الديدلوك ونموذج النظام

📍 الجزء الأول: System Model (نموذج النظام)

1️⃣ المقدمة

🔑 لماذا نموذج النظام؟

قبل فهم Deadlock:

```text
يجب أن نفهم:
- كيف يعمل النظام؟
- ما هي الموارد؟
- كيف تُستخدم؟
```

النموذج:

```text
النظام = Threads + Resources
Threads: تتنافس على Resources
Resources: محدودة ✅
```

2️⃣ الموارد (Resources)

🔑 التعريف

Resource:

- **أي شيء** يحتاجه Thread للتنفيذ
- **محدود** في الكمية
- **يُطلب** و **يُحرر**

أمثلة الموارد:

```text
1. Hardware Resources:
   - CPU cycles (دورات المعالج)
   - Memory pages
   - I/O Devices (Printer, Scanner, Disk)

2. Software Resources:
   - Files (ملفات)
   - Database records
   - Network sockets
```

```
3. Synchronization Tools:
- Mutex locks ✅
- Semaphores ✅
- Monitors
```

⭐ **مهم جداً:** Mutex و Semaphores **موارد** أيضاً!

```text
وهي المصدر الأكثر شيوعاً للـ Deadlock! ⚠️
```

📊 Resource Types vs Instances

Resource Type (نوع المورد):

- **فئة** من الموارد المتطابقة
- مثل: "CPU"، "Printer"، "Mutex"

Resource Instance (نسخة المورد):

- **نسخة محددة** من النوع
- كل Type قد يحتوي **عدة Instances**

المثال:

```text
Resource Type: CPU
- Instance 1: CPU Core 0
- Instance 2: CPU Core 1
→ CPU Type له نسختان ✅

Resource Type: Printer
- Instance 1: HP Printer
- Instance 2: Canon Printer
- Instance 3: Epson Printer
→ Printer Type له 3 نسخ ✅

Resource Type: Mutex_Lock_A
- Instance 1: القفل الوحيد
→ Mutex Type له نسخة واحدة ✅
```

3️⃣ دورة حياة استخدام المورد

🔄 الخطوات الثلاث

كل Thread يتبع هذا التسلسل:

1. Request (الطلب):

```text
Thread يطلب المورد

إذا متاح:
  ✅ يُمنح فوراً

إذا غير متاح:
  ⏳ Thread ينتظر (Wait)
```

2. Use (الاستخدام):

```text
Thread يعمل على المورد
- القراءة
- الكتابة
- التنفيذ
```

3. Release (الإطلاق):

```text
Thread يحرر المورد
→ المورد يصبح متاحاً للآخرين ✅
```

System Calls ⚠️

Request و Release قد تحتاج System Calls:

```text
Request:
- open() للملفات
- request() للأجهزة
- allocate() للذاكرة

Release:
- close() للملفات
- release() للأجهزة
- free() للذاكرة
```

Synchronization Tools:

```text
Request:
- pthread_mutex_lock()
- sem_wait()

Release:
- pthread_mutex_unlock()
- sem_post()
```

4️⃣ خصائص الموارد

📊 القواعد المهمة

1. Mutual Exclusion (لبعض الموارد):

```text
بعض الموارد لا يمكن مشاركتها:
- Mutex lock (واحد فقط يحمله)
- Printer (واحد فقط يطبع)

بعضها قابل للمشاركة:
- Read-only file (عدة threads تقرأ)
```

2. Hold and Wait:

```text
Thread يحمل مورد
+ ينتظر مورد آخر
```

3. No Preemption:

```text
لا يمكن انتزاع المورد عنوة ✅
Thread يحرره طوعاً فقط [web:286]
```

⭐ **للحفظ:**

- **No Preemption** = المورد يُحرر **voluntarily** فقط
- لا يُنتزع عنوة من Thread

📊 ملخص System Model

```text
System Model:
  - Threads تتنافس على Resources
  - Resources محدودة

Resources:
  - Types (أنواع)
  - Instances (نسخ)
  - Hardware + Software + Sync Tools

دورة الحياة:
  Request → Use → Release

خصائص:
  - No Preemption (طوعي فقط)
  - Mutual Exclusion بعضها
```

📍 الجزء الثاني: Deadlock Definition

5️⃣ ما هو الديدلوك؟

🔑 التعريف البسيط

Deadlock (الجمود/الوقوف التام):

```text
حالة يتوقف فيها عدة Threads عن التقدم للأبد
لأن كل واحد ينتظر مورد يمتلكه آخر
```

مثال بسيط:

```text
Thread 1:
  - يحمل Resource A
  - ينتظر Resource B ⏳

Thread 2:
  - يحمل Resource B
  - ينتظر Resource A ⏳

النتيجة: كلاهما ينتظر للأبد! ⚠️
```

⭐ **للحفظ:** Deadlock = **انتظار دائري متبادل - لا أحد يتقدم**

🔑 التعريف الدقيق

التعريف الرسمي:

```text
مجموعة من Processes (أو Threads) في حالة Deadlock
إذا كان كل Process في المجموعة ينتظر حدثاً
لا يمكن أن يحدث إلا بواسطة Process آخر في نفس المجموعة
```

الأحداث المقصودة:

- **الحصول** على مورد (Acquire)
- **إطلاق** مورد (Release)

⭐ **للحفظ:** كل واحد **ينتظر الآخر - حلقة مفرغة**

6️⃣ كيف يحدث؟

🔄 السيناريو

السياق: Multiprogramming Environment

```text
عدة Threads تعمل بالتوازي
Resources محدودة
المنافسة على الموارد
```

الخطوات:

1. Thread يطلب مورد غير متاح:

```c
Thread T1:
pthread_mutex_lock(&lock_B);  // lock_B محجوز! ❌
// T1 → Waiting State ⏳
```

2. المورد محجوز من Thread آخر:

```c
Thread T2:
// lock_B يحمل ✅
```

3. T2 أيضاً ينتظر:

```c
Thread T2:
// lock_B يحمل
pthread_mutex_lock(&lock_A);  // lock_A محجوز من T1! ❌
// T2 → Waiting State ⏳
```

4. Deadlock!

```text
T1: يحمل lock_A, ينتظر lock_B ⏳
T2: يحمل lock_B, ينتظر lock_A ⏳

لا أحد يمكنه التقدم! ⚠️
```

8️⃣ فشل دورة الحياة

🔄 Request-Use-Release

الوضع الطبيعي:

```text
✅ T1: Request(A) → Use(A) → Release(A)
✅ Use(A) → (Release بعد) → ⏳ T2: Request(A)
```

مع Deadlock:

```text
⏳ T1: Request(A) ✅ → Request(B) ← عالق!
                    ❌ Use لا
                    ❌ Release لا

⏳ T2: Request(B) ✅ → Request(A) ← عالق!
                    ❌ Use لا
                    ❌ Release لا

⚠️ الدورة فشلت!
```

⭐ النتيجة:

```text
لا أحد يصل لـ Release
→ لا أحد يتقدم
→ Deadlock [web:281]
```

9️⃣ الخصائص الأساسية

📊 صفات Deadlock

1. Permanent (دائم):

```text
❌ لا يحل نفسه
يستمر للأبد بدون تدخل
```

2. Circular Wait (انتظار دائري):

```text
T1 ينتظر T2 ←
T2 ينتظر T3 ←
...
Tn ينتظر T1 ←
حلقة! ⭕ ←
```

3. Multiple Processes:

```text
على الأقل اثنان ✅
(يمكن أكثر)
```

4. Multiple Resources:

```text
على الأقل اثنان ✅
(يمكن أكثر)
```

🔟 تأثيرات Deadlock

⚠️ العواقب

1. System Hang:

```text
البرنامج يتوقف تماماً
لا استجابة
```

2. Resource Waste:

```text
الموارد محجوزة
لكن غير مستخدمة ❌
```

3. User Frustration:

```text
المستخدم ينتظر... ⏳
لا شيء يحدث
```

4. Requires Intervention:

```text
يحتاج تدخل يدوي:
- Kill processes
- Restart system
```

📊 الملخص الكامل

System Model

```text
Components:
- Threads (تتنافس)
- Resources (محدودة)

Resources:
- Types (أنواع)
- Instances (نسخ)
- Hardware/Software/Sync Tools

دورة الحياة:
Request → Use → Release

No Preemption:
- يُحرر طوعاً فقط ✅
```

Deadlock

```text
Definition:
- مجموعة من Threads
- كل واحد ينتظر الآخر
- للأبد ⚠️

كيف يحدث:
- Thread يحمل R1, ينتظر R2
- Thread آخر يحمل R2, ينتظر R1
- Circular Wait
```

الخصائص:

- Permanent (دائم)
- Circular (دائري)
- Multiple (متعدد)

التأثير:

- System Hang
- Resource Waste
- Requires Intervention

الشروط الضرورية للديدلوك

🔑 القاعدة الذهبية

⚠️ الشروط الأربعة الضرورية

القاعدة الأساسية:

```text
Deadlock يحدث إذا وفقط إذا:
  تحققت الأربعة شروط معاً في نفس الوقت! ✅

إذا شرط واحد غير متحقق:
  لا يمكن حدوث Deadlock! ✅
```

الشروط الأربعة:

```text
1. Mutual Exclusion (الاستبعاد المتبادل)
2. Hold and Wait (الاحتفاظ والانتظار)
3. No Preemption (عدم التجريد)
4. Circular Wait (الانتظار الدائري)
```

⭐ للحفظ:

```text
4 شروط + معاً + simultaneously = Deadlock ⚠️
شرط واحد مفقود = لا Deadlock ✅
```

📍 الجزء الأول: Mutual Exclusion (الاستبعاد المتبادل)

1️⃣ التعريف

🔑 المفهوم الأساسي

Mutual Exclusion:

```text
على الأقل مورد واحد في النظام:
  - غير قابل للمشاركة (Non-sharable) ❌
  - Thread واحد فقط يستخدمه في وقت معين
  - جميع الآخرين يجب أن ينتظروا ⏳
```

الفكرة الجوهرية:

```text
المورد = حصري (Exclusive)
لا يمكن مشاركته بين عدة Threads في نفس اللحظة
```

⭐ **للحفظ:** Mutual Exclusion = **مورد حصري** - واحد فقط في كل مرة

📝 أمثلة توضيحية

مثال 1: Mutex Lock

```text
الموقف:
- Thread 1 يأخذ القفل ✅
- Thread 2 يحاول أخذ نفس القفل ⏳

النتيجة:
- Thread 2 يُحجب ويدخل في الانتظار
- لا يمكن لاثنين حمل نفس Mutex

→ Mutual Exclusion ✅
```

مثال 2: الطابعة (Printer)

```text
الموقف:
- Thread 1 يطبع مستند ✅
- Thread 2 يريد الطباعة على نفس الطابعة ⏳

لماذا الانتظار؟
- لو اثنان يطبعان معاً:
  * الإخراج سيختلط ❌
  * المستند الأول سيفسد ❌
  * المستند الثاني سيفسد ❌

الحل:
- Thread 2 ينتظر حتى Thread 1 ينتهي

→ Mutual Exclusion ✅
```

📊 تصنيف الموارد

موارد Non-sharable (تحتاج Mutual Exclusion):

```text
✅ Mutex locks
✅ Binary Semaphores (قيمة 1)
✅ Printer (طابعة)
✅ Tape drive (محرك شريط)
✅ Writer lock on file (قفل كتابة على ملف)
✅ CD-ROM writer (أثناء الكتابة)
✅ Scanner (ماسح ضوئي)
```

موارد Sharable (لا تحتاج Mutual Exclusion):

```text
✅ Read-only files (ملفات قراءة فقط)
   - عدة Threads يمكنهم القراءة معاً

✅ CPU cycles مع Time-sharing
   - جدولة تبادلية

✅ Shared memory للقراءة
   - القراءة فقط = آمنة للمشاركة

✅ Counting Semaphore (قيمة > 1)
   - يسمح لعدة threads بالدخول
```

⭐ المفتاح:

```text
✅ Sharable resources → لا تسبب Deadlock
⚠️ Non-sharable resources → قد تسبب Deadlock
```

✅ متى يتحقق الشرط؟

الشرط متحقق عندما:

```text
✅ يوجد مورد واحد على الأقل غير قابل للمشاركة
✅ Thread واحد فقط يستطيع استخدامه
✅ الآخرون يُحجبون عند الطلب
```

الشرط غير متحقق عندما:

```text
❌ كل الموارد قابلة للمشاركة (مثل Read-only)
❌ عدة Threads يستطيعون الوصول معاً
❌ لا حجب عند الطلب
```

🎯 أهمية الشرط

لماذا ضروري للديدلوك؟

```text
بدون Mutual Exclusion:
  - كل Thread يأخذ ما يحتاج مباشرة ✅
  - لا انتظار ⏳
  - لا يحمل ويطلب المزيد

→ لا يمكن حدوث Deadlock ✅
```

مع Mutual Exclusion:

```text
Thread يطلب مورد حصري:
  - إذا متاح: يأخذه ✅
  - إذا محجوز: ينتظر ⏳

→ بداية سلسلة الانتظار
→ الشرط الأول للديدلوك متحقق ⚠️
```

📍 الجزء الثاني: Hold and Wait (الاحتفاظ والانتظار)

2️⃣ التعريف

🔑 المفهوم الأساسي

Hold and Wait:

```text
يوجد Thread بالمواصفات التالية:
1. يحمل (Hold) مورد واحد على الأقل ✅
2. في نفس الوقت ينتظر (Wait) موارد إضافية ⏳
3. الموارد المنتظرة محجوزة من threads أخرى
```

الحالة:

```text
Thread في حالة "ثنائية":
  - من جهة: يملك موارد ✅
  - من جهة أخرى: ينتظر موارد أخرى ⏳

→ Hold + Wait معاً
```

📝 أمثلة توضيحية

مثال 1: القفلان (Two Locks)

```text
الموقف:
  Thread T1:
    - أخذ lock_A بنجاح ✅
    - الآن يحمل lock_A
    - يحاول أخذ lock_B ⏳
    - لكن lock_B محجوز من T2

الحالة:
  - Hold: lock_A ✅
  - Wait: lock_B ⏳

→ Hold and Wait ✅
```

🔍 التحليل العميق

لماذا يحدث Hold and Wait؟

```text
السبب الرئيسي:
  - الموارد تُطلب تدريجياً (Incrementally)
  - ليس كلها مرة واحدة

النتيجة:
  - Thread يأخذ بعض الموارد
  - ثم يطلب المزيد أثناء حمل الأولى

→ Hold and Wait
```

البديل الذي يمنعه:

```text
طلب كل الموارد مرة واحدة:
- Thread يطلب A و B معاً
- إما يأخذ الاثنين ✅
- أو لا يأخذ شيء ❌

→ لا Hold and Wait ✅
→ لكن قد يكون غير عملي
```

✅ متى يتحقق الشرط؟

الشرط متحقق عندما:

```text
✅ Thread يحمل مورد R1 على الأقل
+ ✅ Thread ينتظر مورد R2
+ ✅ R2 محجوز من thread آخر
+ ✅ في نفس اللحظة
```

الشرط غير متحقق عندما:

```text
❌ Thread لا يحمل أي مورد ويطلب
   (فقط Wait بدون Hold)

❌ Thread يحمل كل الموارد المطلوبة مسبقاً
   (Hold فقط بدون Wait)

❌ Thread يحرر كل ما يحمل قبل طلب المزيد
   (Hold تم Release تم Wait منفصلين)
```

📍 الجزء الثالث: No Preemption (عدم التجريد)

3️⃣ التعريف

🔑 المفهوم الأساسي

No Preemption:

```text
الموارد لا يمكن انتزاعها عنوة من Thread! ❌

خصائص المورد:
- يُحرر طوعاً (Voluntarily) فقط ✅
- من Thread الذي يحمله
- بعد إكمال المهمة

نظام التشغيل:
- لا يستطيع سحب المورد بالقوة ❌
- يجب انتظار التحرير الطوعي
```

الفكرة:

```text
Thread يملك المورد = سيادة كاملة عليه
لا أحد (حتى OS) يستطيع انتزاعه
```

📝 أمثلة توضيحية

مثال 1: Mutex Lock

```text
الموقف:
- Thread T1 يحمل Mutex ✅
- Thread T2 ينتظر نفس Mutex ⏳
- T2 له أولوية عالية جداً!

ماذا يفعل OS؟
- OS لا يستطيع انتزاع Mutex من T1 ❌
- OS لا يملك دالة "force_unlock" ❌
- T2 يجب أن ينتظر حتى T1 يحرر طوعاً ✅

→ No Preemption ✅
```

🔄 المقارنة مع Preemptive

المورد Preemptive (يمكن انتزاعه):

```text
مثال: CPU
  - Thread يعمل على CPU ✅
  - Interrupt يحدث
  - OS يوقف Thread (Context Switch)
  - Thread آخر يأخذ CPU ✅

→ Preemptive ✅
→ لا مشكلة! البيانات آمنة
```

المورد No Preemption (لا يُنتزع):

```text
مثال: Mutex Lock
  - Thread يحمل Mutex ✅
  - Thread آخر ينتظر
  - OS لا يستطيع إزالة Thread من Mutex ❌
  - Thread يحرره بنفسه فقط ✅

→ No Preemption ✅
→ ضروري! لحماية البيانات
```

⭐ الفرق الجوهري:

```text
CPU:
  - يمكن حفظ الحالة (Context)
  - تم استرجاعها لاحقاً
  - Preemptive آمن ✅

Mutex/Locks:
  - لا يمكن "حفظ" حالة Critical Section
  - البيانات المشتركة في خطر
  - No Preemption ضروري ✅
```

✅ متى يتحقق الشرط؟

الشرط متحقق عندما:

```text
✅ الموارد تُحرر طوعاً فقط
✅ OS لا يملك آلية "force release"
✅ Thread يحرر عند انتهاء المهمة فقط
```

الشرط غير متحقق عندما:

```text
❌ OS يستطيع انتزاع الموارد عنوة
❌ يوجد timeout mechanism (تحرير تلقائي)
❌ OS يستطيع "سحب" المورد بالقوة
```

🎯 أمثلة على كسر الشرط

الطريقة 1: Timeout

```text
الفكرة:
  - Thread يحاول أخذ lock
  - إذا لم يحصل عليه خلال وقت معين
  - يتنازل عن كل ما يحمل ✅
  - يحاول لاحقاً

→ نوع من "الانتزاع الذاتي" ✅
→ يكسر No Preemption
```

الطريقة 2: Trylock

```text
الفكرة:
  - Thread يحاول أخذ lock
  - إذا فشل: لا ينتظر
  - يحرر ما يحمل طوعاً ✅
  - يحاول المجموعة كاملة لاحقاً

→ تحرير طوعي استباقي ✅
→ يكسر No Preemption (نوعاً ما)
```

📍 الجزء الرابع: Circular Wait (الانتظار الدائري)

4️⃣ التعريف

🔑 المفهوم الأساسي

Circular Wait:

```text
مجموعة من Threads المنتظرة:
{T0, T1, T2, ..., Tn}

بحيث تتشكل حلقة:
- T0 ينتظر مورد يحمله T1 ⏳
- T1 ينتظر مورد يحمله T2 ⏳
- T2 ينتظر مورد يحمله T3 ⏳
- ...
- Tn ينتظر مورد يحمله T0 ⏳

→ حلقة مقفلة! ⭕
→ لا بداية ولا نهاية
→ كل واحد ينتظر الآخر
```

الفكرة:

```text
انتظار دائري = Circular Dependency
كل واحد يعتمد على التالي
والأخير يعتمد على الأول
```

📝 أمثلة توضيحية

مثال 1: حلقة بسيطة (2 Threads)

```text
السيناريو:
- Thread T0 يحمل Resource A ✅
- Thread T0 ينتظر Resource B ⏳

- Thread T1 يحمل Resource B ✅
- Thread T1 ينتظر Resource A ⏳
```

```
التحليل:
  T0 → ينتظر T1 (لأن T1 يحمل B)
  T1 → ينتظر T0 (لأن T0 يحمل A)

الحلقة:
  T0 → T1 → T0
  ⭕ حلقة مقفلة!

→ Circular Wait ✅
```

🔍 العلاقة مع Hold and Wait

ملاحظة مهمة جداً:

```text
Circular Wait ضمنياً يشمل Hold and Wait! ✅
```

الشرح:

```text
في Circular Wait:
  كل Thread في الحلقة:
    - يحمل مورد ✅
    - وينتظر مورد آخر ⏳

→ هذا تعريف Hold and Wait! ✅
```

التحليل:

```text
T0: يحمل A + ينتظر B
  └── Hold and Wait! ✅

T1: يحمل B + ينتظر A
  └── Hold and Wait! ✅

→ كل Thread في حالة Hold and Wait
→ Circular Wait يتضمن Hold and Wait
```

⭐ لماذا نفصلهم في الدراسة؟

```text
أسباب أكاديمية:
- كل شرط يُدرس بمعزل ✅
- لفهم أعمق لكل شرط
- لدراسة كيفية كسر كل شرط منفرداً

في الواقع:
- Circular Wait → يحتوي Hold and Wait تلقائياً ✅
- لا يمكن Circular Wait بدون Hold and Wait
```

✅ متى يتحقق الشرط؟

الشرط متحقق عندما:

```text
✅ يوجد سلسلة من Threads
✅ كل Thread ينتظر التالي
✅ الأخير ينتظر الأول
✅ تتشكل حلقة مقفلة ⭕
```

الشرط غير متحقق عندما:

```text
❌ الانتظار في ترتيب خطي (لا حلقة)
❌ Thread واحد ينتظر فقط (لا حلقة)
❌ لا يوجد انتظار متبادل
```

🛡️ كسر Circular Wait

الطريقة الأكثر فعالية: ترتيب الموارد

المفهوم:

```text
نعطي كل مورد رقم فريد:
    R0 = المورد 0
    R1 = المورد 1
    R2 = المورد 2
    ...

القاعدة:
كل Thread يجب أن يطلب الموارد بترتيب تصاعدي! ✅
```

مثال تطبيقي:

```text
الموارد:
    lock_A → رقم 1
    lock_B → رقم 2
    lock_C → رقم 3

القاعدة:
دائماً اطلب بالترتيب: 1 → 2 → 3

Thread T0:
    1. اطلب lock_A (رقم 1) ✅
    2. اطلب lock_B (رقم 2) ✅
    → ترتيب صحيح ✅

Thread T1:
    1. اطلب lock_A (رقم 1) ✅
    2. اطلب lock_B (رقم 2) ✅
    → ترتيب صحيح ✅

النتيجة:
- T1 سينتظر T0 ينتهي من A و B
- لكن لا حلقة! ❌
- T0 سيحرر، ثم T1 يأخذ
- لا Deadlock ✅
```

لماذا يعمل الترتيب؟

```text
بدون ترتيب:
  T0: A → B
  T1: B → A
  → حلقة ممكنة! ⭕

مع ترتيب:
  T0: A → B
  T1: A → B (نفس الترتيب!)
  → لا حلقة! ✅

السبب:
  - لا يمكن تكوين حلقة
  - الانتظار دائماً في اتجاه واحد (تصاعدي)
  - لا رجوع للخلف
```

📊 الربط الكامل: الشروط الأربعة معاً

5️⃣ كيف يحدث Deadlock؟

🔄 السيناريو المتكامل

الموقف:

```text
لدينا:
  - Thread T0
  - Thread T1
  - Resource A (Mutex)
  - Resource B (Mutex)
```

تسلسل الأحداث:

Thread T0:

```text
1. يطلب Resource A
   → يحصل عليه ✅

تحليل:
  ✅ Mutual Exclusion (A حصري)
  ✅ Hold (T0 يحمل A الآن)
```

```
2. يعمل قليلاً...

3. يطلب Resource B
   → B محجوز من T1 ⏳
   → T0 ينتظر...

تحليل:
✅ Hold and Wait (يحمل A + ينتظر B)
✅ No Preemption (لا يمكن انتزاع A من T0)
```

Thread T1:

```text
1. يطلب Resource B
   → يحصل عليه ✅

تحليل:
✅ Mutual Exclusion (B حصري)
✅ Hold (T1 يحمل B الآن)

2. يعمل قليلاً...

3. يطلب Resource A
   → A محجوز من T0 ⏳
   → T1 ينتظر...

تحليل:
✅ Hold and Wait (يحمل B + ينتظر A)
✅ No Preemption (لا يمكن انتزاع B من T1)
✅ Circular Wait (T0↔T1) ⭕
```

النتيجة النهائية:

```text
الأربعة شروط متحققة:

✅ 1. Mutual Exclusion
   - A و B كلاهما حصريان
   - لا يمكن مشاركتهما

✅ 2. Hold and Wait
   - T0: يحمل A, ينتظر B
   - T1: يحمل B, ينتظر A
```

```
✅ No Preemption .3
   - OS لا يستطيع انتزاع A من T0
   - OS لا يستطيع انتزاع B من T1
   - تحرير طوعي فقط

✅ Circular Wait .4
   - T0 ينتظر T1
   - T1 ينتظر T0
   - حلقة مقفلة ⭕

→ الأربعة شروط + معاً = Deadlock! ⚠️⚠️⚠️
```

6️⃣ استراتيجيات كسر الديدلوك

🔧 كسر أي شرط = منع Deadlock

القاعدة:

```text
إذا كسرنا شرط واحد فقط:
→ الأربعة لن تتحقق معاً
→ لا يمكن حدوث Deadlock ✅
```

1️⃣ كسر Mutual Exclusion

الفكرة:

```text
اجعل الموارد قابلة للمشاركة
→ عدة Threads يستخدمون نفس المورد معاً
```

مثال:

```text
بدلاً من:
❌ Write lock (حصري)

استخدم:
✅ Read-only sharing
   - عدة Threads يقرؤون معاً
   - لا حجب
```

التقييم:

```text
الإيجابيات:
✅ يمنع Deadlock بشكل جذري

السلبيات:
❌ غير ممكن لمعظم الموارد
❌ طبيعة المورد تحتاج Exclusivity
❌ مثلاً: Printer لا يمكن مشاركته!

الصعوبة: ⭐⭐⭐⭐⭐ (الأصعب!)
```

2️⃣ كسر Hold and Wait

الطريقة 1: احجز الكل مرة واحدة

```text
المفهوم:
Thread يطلب كل الموارد المطلوبة دفعة واحدة

إما:
- يحصل على الجميع ✅ → يكمل
- أو لا يحصل على شيء ❌ → ينتظر الكل

لا Hold and Wait:
- لا يحمل جزء وينتظر جزء
- إما كل شيء أو لا شيء
```

مثال:

```text
بدلاً من:
1. اطلب A → خذه ✅
2. اطلب B → انتظر ⏳
→ Hold and Wait ❌

استخدم:
1. اطلب A و B معاً
2. إذا كلاهما متاح → خذهما ✅
3. إذا أحدهما محجوز → لا تأخذ شيء ❌
→ لا Hold and Wait ✅
```

الطريقة 2: حرر قبل طلب المزيد

```text
المفهوم:
  Thread يحرر كل ما يحمل قبل طلب موارد جديدة

الخطوات:
  1. اطلب A → خذه ✅
  2. استخدم A
  3. حرر A ✅
  4. اطلب B → خذه ✅
  5. استخدم B
  6. حرر B ✅

لا Hold and Wait:
  - لا يحمل A أثناء طلب B
```

التقييم:

```text
الإيجابيات:
  ✅ يمنع Deadlock
  ✅ بسيط نظرياً

السلبيات:
  ❌ قد يكون غير عملي
  ❌ بعض البروتوكولات تحتاج حمل عدة موارد
  ❌ إهدار الموارد (طلب الكل وقد لا نستخدمها كلها)

الصعوبة: ⭐⭐⭐ (متوسط)
```

3️⃣ كسر No Preemption

الطريقة 1: Trylock مع Timeout

```text
المفهوم:
  Thread يحاول أخذ الموارد بدون انتظار
  إذا فشل → يحرر كل ما يحمل طوعاً

الخطوات:
  1. خذ A ✅
  2. حاول أخذ B (trylock)
  3. إذا نجح:
     → أكمل ✅
  4. إذا فشل:
     → حرر A طوعاً ✅
     → انتظر قليلاً
     → حاول مرة أخرى

النتيجة:
  - "انتزاع ذاتي" ✅
  - Thread يحرر قبل أن يُجبر
```

الطريقة 2: Timeout Mechanism

```text
المفهوم:
  إذا Thread انتظر طويلاً
  → يتنازل تلقائياً عن كل ما يحمل

الآلية:
  1. Thread يطلب B
  2. ينتظر... ⏳
  3. إذا انتهت المهلة (مثلاً 5 ثواني)
     → يحرر كل ما يحمل ✅
     → يحاول لاحقاً
```

التقييم:

```text
الإيجابيات:
✅ عملي جداً
✅ يكسر No Preemption بذكاء

السلبيات:
❌ قد يسبب Livelock (كلهم يحررون ويحاولون)
❌ overhead من المحاولات المتكررة

الصعوبة: ⭐⭐ (سهل نسبياً)
```

4️⃣ كسر Circular Wait

الطريقة: ترتيب الموارد (Resource Ordering)

المفهوم:

```text
نعطي كل مورد رقم فريد
كل Thread يطلب الموارد بترتيب تصاعدي فقط
```

الترقيم:

```text
lock_A → رقم 1
lock_B → رقم 2
lock_C → رقم 3
...
```

القاعدة الذهبية:

```text
دائماً اطلب المورد الأصغر رقماً أولاً!
```

التقييم:

```text
الإيجابيات:
✅ فعال جداً
✅ سهل التطبيق
✅ لا overhead كبير
✅ الأكثر استخداماً عملياً

السلبيات:
❌ يحتاج تخطيط مسبق للترقيم
❌ قد يكون صعباً في الأنظمة المعقدة

الصعوبة: ⭐ (الأسهل!)
```

📊 الجدول المقارن الشامل

الشرط	التعريف	أمثلة	كيفية الكسر	الصعوبة
1. Mutual Exclusion	مورد حصري واحد فقط	Mutex, Printer	اجعله قابل للمشاركة	⭐⭐⭐⭐⭐ أصعب
2. Hold and Wait	يحمل + ينتظر معاً	يحمل A ينتظر B	احجز الكل مرة واحدة	⭐⭐⭐ متوسط
3. No Preemption	لا انتزاع عنوة	تحرير طوعي فقط	trylock + timeout	⭐⭐ سهل نسبياً
4. Circular Wait	حلقة انتظار ⭕	T0↔T1	ترتيب الموارد	⭐ أسهل

ورقة الغش النهائية: شروط الديدلوك (Deadlock)

القاعدة الذهبية:

4 شروط + معاً (Simultaneously) = Deadlock ⚠

شرط واحد مفقود = لا يوجد Deadlock ✔

---

1 الاستبعاد المتبادل (Mutual Exclusion)

- **المفهوم:** مورد غير قابل للمشاركة، يستخدمه طرف واحد فقط.
- **أمثلة:** الطابعة (Printer)، القفل الحصري (Mutex).
- **ليس منها:** الملفات للقراءة فقط (Read-only).
- **كسره:** ✕ **صعب جداً** (بسبب طبيعة المورد الفيزيائية).
- **الصعوبة:** ☆☆☆☆☆

---

2 الاحتفاظ والانتظار (Hold and Wait)

- **المفهوم:** الخيط (Thread) يحجز موردًا وينتظر موردًا آخر في نفس الوقت.
- **مثال:** يحمل القفل A وينتظر القفل B.
- **كسره:** ✔ احجز جميع الموارد المطلوبة دفعة واحدة.
  ✔ حرر ما تملك قبل أن تطلب موارد جديدة.
- **الصعوبة:** ☆☆☆

---

3 عدم التجريد (No Preemption)

- **المفهوم:** لا يمكن انتزاع المورد عنوة؛ التحرير يكون طوعياً فقط.
- **أمثلة:** أقفال قواعد البيانات، الطابعة.
- **ليس منها:** المعالج (CPU) لأنه يدعم المقاطعة.
- **كسره:** ✔ استخدام (trylock) مع مؤقت (timeout).
  ✔ التنازل الطوعي عن الموارد إذا لم تتوفر البقية.
- **الصعوبة:** ☆☆

---

4 الانتظار الدائري (Circular Wait)

- **المفهوم:** حلقة مغلقة (T0 ينتظر T1، و T1 ينتظر T0).
- **ملاحظة:** هذا الشرط يتضمن "الاحتفاظ والانتظار" ضمنياً.
- **كسره:** ✔ **ترتيب الموارد (Resource Ordering):** إعطاء رقم فريد لكل مورد.
  ✔ الطلب دائماً يكون بترتيب تصاعدي (من الرقم الأصغر للأكبر).
- **الصعوبة:** ☆ **(الأسهل والأكثر فعالية عملياً!)**

🛠️ **كيف نكسر الديدلوك؟ (الملخص)**

الشرط	طريقة الكسر (الحل)	السهولة
الانتظار الدائري	ترتيب الموارد تصاعدياً (1 ← 2 ← 3)	الأسهل والأفضل ✅
عدم التجريد	استخدام (Trylock) والانسحاب عند الفشل	سهل
الاحتفاظ والانتظار	طلب كل الموارد في البداية فقط	متوسط
الاستبعاد المتبادل	جعل المورد قابلاً للمشاركة	شبه مستحيل

طرق التعامل مع الديدلوك

🎯 الاستراتيجيات الثلاثة

النظام أمامه 3 خيارات للتعامل مع مشكلة الديدلوك:

```text
1. التجاهل (Ignore) 🙈
2. الضمان (Ensure it never happens) 🛡️
3. السماح والمعالجة (Allow & Fix) 🔍
```

📍 الاستراتيجية الأولى: Ignore the Problem

المفهوم: خوارزمية النعامة (Ostrich Algorithm)

النظام يتجاهل المشكلة تماماً ويتظاهر أن الديدلوك لن يحدث.

التبرير

- **الندرة**: الديدلوك نادر الحدوث جداً
- **التكلفة العالية**: حلول المنع والاكتشاف مكلفة وتقلل الأداء
- **الحساب الاقتصادي**: تكلفة التجاهل < تكلفة الحلول
- **البساطة**: لا تعقيد في التصميم، لا overhead على الأداء

الاستخدام

معظم أنظمة التشغيل العامة تستخدم هذه الاستراتيجية:

- **Windows, Linux, UNIX**: لا تمنع ولا تكتشف الديدلوك
- **النتيجة عند حدوث Deadlock**: النظام يتجمد (Hang)
- **الحل**: المستخدم يعمل Restart أو يقتل العمليات يدوياً

متى تستخدم؟

مناسب لـ:

- الأنظمة العامة (General-purpose OS)
- عندما الديدلوك نادر جداً
- عندما المستخدم يستطيع التدخل

غير مناسب لـ:

- الأنظمة الحرجة (أنظمة طبية، طيران، نووية)
- Real-time systems
- الخوادم الحرجة التي لا يمكن إيقافها

📍 الاستراتيجية الثانية: Ensure it Never Happens

تنقسم إلى نوعين: **Prevention** و **Avoidance**

🛡️ أولاً: Deadlock Prevention

المفهوم

منع الديدلوك عن طريق **كسر واحد على الأقل** من الشروط الأربعة الضرورية.

الطرق الأربعة

1. كسر Mutual Exclusion

الفكرة: اجعل الموارد قابلة للمشاركة

التقييم: غير عملي لمعظم الموارد (Mutex locks لا يمكن مشاركتها!)

---

2. كسر Hold and Wait

الطريقة الأولى: احجز كل الموارد مرة واحدة (All-or-Nothing)

- Process يطلب كل موارده قبل البدء
- إما يحصل على الجميع أو لا شيء

السلبيات:

- إهدار الموارد (Low resource utilization)
- Starvation ممكنة
- Process قد لا يعرف احتياجاته مسبقاً

الطريقة الثانية: حرر قبل طلب موارد جديدة

- Process يحرر كل ما يحمل قبل طلب المزيد

السلبيات:

- قد يكون مستحيلاً (بعض البروتوكولات تحتاج حمل عدة موارد)
- فقدان العمل السابق (Inconsistency)

---

3. كسر No Preemption

الفكرة: السماح بانتزاع الموارد عنوة

الآلية: عندما Process ينتظر مورد محجوز، OS ينتزع منه ما يحمل

التقييم:

- عملي فقط لموارد معينة (CPU, Memory)
- غير ممكن للـ Locks والأجهزة
- يحتاج Rollback mechanism

---

4. كسر Circular Wait ⭐

الفكرة: Resource Ordering (ترتيب الموارد)

الآلية:

1. أعط كل مورد رقم فريد
2. كل Process يطلب الموارد بترتيب تصاعدي فقط

مثال توضيحي:

```text
الترقيم:
Mutex_A = رقم 5
Mutex_B = رقم 12
Scanner = رقم 8

الترتيب الصحيح:
✅ Process: A(5) → Scanner(8) → B(12)

الترتيب الخاطئ:
❌ Process: B(12) → A(5)  (رجوع للخلف ممنوع!)

لماذا يعمل؟
لا يمكن تكوين حلقة دائرية عندما الجميع يطلبون بنفس الاتجاه
```

التقييم:

- **الأكثر فعالية وعملية** من بين الأربعة
- منخفض التكلفة، بسيط التطبيق
- **الحل العملي الوحيد** المستخدم فعلياً
- السلبية الوحيدة: يحتاج تخطيط مسبق للترقيم

🧠 ثانياً: Deadlock Avoidance

المفهوم

النظام يتخذ **قرارات ذكية** بناءً على **معلومات مسبقة** لتجنب الدخول في حالات خطرة.

الفرق عن Prevention

- **Prevention**: قاسٍ وجامد، يكسر الشروط
- **Avoidance**: ذكي ومرن، يستخدم معلومات مسبقة

المتطلبات

كل Process يجب أن يعلن مسبقاً:

- **Maximum Need**: أقصى احتياج من كل مورد

المفاهيم الأساسية

Safe State (الحالة الآمنة)

النظام في Safe State إذا كان يوجد **تسلسل آمن** (Safe Sequence) بحيث كل Process يستطيع الحصول على احتياجاته القصوى وينتهي بنجاح.

Unsafe State (الحالة غير الآمنة)

لا يوجد تسلسل آمن - قد يحدث Deadlock (ليس بالضرورة لكن ممكن).

العلاقة

```text
Safe State → لا Deadlock (ضمان 100%)
Unsafe State → قد يحدث Deadlock
Deadlock → دائماً Unsafe
```

خوارزمية المصرفي **(Banker's Algorithm)**

التشبيه: مثل المصرفي الذي يوافق على قرض فقط إذا كان يستطيع تلبية كل الطلبات القصوى المستقبلية.

البيانات المطلوبة:

- **Available**: الموارد المتاحة حالياً
- **Max**: الحد الأقصى لاحتياج كل Process
- **Allocation**: ما يحمله كل Process حالياً
- **Need**: الحاجة المتبقية = Max - Allocation

آلية العمل:

عندما Process يطلب موارد:

1. تحقق: هل الطلب ≤ Need؟
2. تحقق: هل الطلب ≤ Available؟
3. خصص الموارد **تجريبياً**
4. نفذ Safety Algorithm: هل النظام سيبقى Safe؟
5. إذا Safe → اقبل الطلب، وإلا → ارفض وانتظر

Safety Algorithm باختصار:

- ابحث عن Process يستطيع الانتهاء (Need ≤ Available)
- افترض أنه انتهى وحرر موارده
- كرر حتى تنتهي كل العمليات
- إذا نجحت → Safe، وإلا → Unsafe

التقييم:

الإيجابيات:

- يمنع Deadlock بضمان 100%
- أكثر مرونة من Prevention

السلبيات:

- يحتاج معلومات مسبقة (Maximum needs)
- عدد Processes ثابت (لا Dynamic creation)
- عدد Resources ثابت
- Overhead عالي
- الافتراضات غير واقعية

الحكم: نظرياً ممتاز، عملياً محدود الاستخدام (أنظمة صغيرة ومحددة فقط).

📍 الاستراتيجية الثالثة: Detection & Recovery

المفهوم

النظام **يسمح بحدوث Deadlock**، لكنه يكتشفه ويعالجه.

🔍 أولاً: Detection (الاكتشاف)

الآلية

النظام يشغل **خوارزمية كشف** بشكل دوري للبحث عن Deadlock.

طريقة الكشف

للموارد أحادية النسخة **(Single Instance)**

استخدام **Wait-for Graph**:

- رسم بياني: كل عقدة = Process
- سهم من P1 إلى P2 = P1 ينتظر P2
- **إذا وُجدت دورة (Cycle) ← يوجد Deadlock**

للموارد متعددة النسخ **(Multiple Instances)**

استخدام خوارزمية مشابهة لـ Banker's:

- محاولة إيجاد تسلسل يمكن فيه إنهاء كل العمليات
- إذا فشلت ← يوجد Deadlock

🔧 ثانياً: Recovery (المعالجة)

عند اكتشاف Deadlock، النظام يجب أن **يكسر الحلقة**.

الطريقة الأولى: Process Termination (إنهاء العمليات)

الخيار 1: إنهاء الكل (Abort All)

- اقتل كل العمليات في الديدلوك دفعة واحدة
- **السلبية**: مكلف جداً، خسارة كل العمل المنجز

الخيار 2: إنهاء تدريجي (Abort One at a Time)

- اقتل عملية واحدة في كل مرة
- بعد كل عملية، نفذ Detection مرة أخرى
- إذا لا يزال Deadlock → اقتل عملية أخرى
- كرر حتى يزول الديدلوك

معايير اختيار الضحية:

- الأولوية: اقتل العمليات منخفضة الأولوية
- الوقت المستهلك: اقتل من عمل أقل (خسارة أقل)
- الموارد المحمولة: اقتل من يحمل موارد كثيرة
- عدد العمليات المتبقية: الأقل تعقيداً

الطريقة الثانية: Resource Preemption (سحب الموارد)

الآلية

- انتزع الموارد **عنوة** من بعض العمليات
- أعطها لعمليات أخرى لكسر الحلقة
- العملية المسحوب منها تُرجع لحالة سابقة آمنة (Rollback)

التحديات

1. اختيار الضحية (Selecting a Victim)

- من أي Process نسحب الموارد؟
- نفس المعايير السابقة: الأولوية، التكلفة، الموارد

2. Rollback

- إرجاع Process لحالة سابقة آمنة
- يحتاج **Checkpointing**: حفظ الحالة بشكل دوري
- **Total Rollback**: الرجوع للبداية تماماً (بسيط لكن مكلف)
- **Partial Rollback**: الرجوع لآخر Checkpoint آمن (أفضل)

3. Starvation

- نفس Process قد يُختار كضحية مراراً وتكراراً
- **الحل**: تتبع عدد المرات، لا تختر نفس الضحية أكثر من X مرة

📊 المقارنة الشاملة

الاستراتيجية	متى تستخدم؟	التكلفة	الفعالية	الاستخدام الفعلي
Ignore	أنظمة عامة، Deadlock نادر	صفر	-	**Windows, Linux**
Prevention	عندما يمكن كسر شرط	منخفضة-متوسطة	عالية	Resource Ordering شائع
Avoidance	أنظمة صغيرة معروفة	عالية	عالية	نادر (أنظمة محددة)
Detection & Recovery	أنظمة تتحمل التوقف المؤقت	متوسطة-عالية	متوسطة	Databases أحياناً

🎯 الخلاصة المركزة

أكثر الطرق استخداماً عملياً:

1. **Ignore** (الأكثر شيوعاً - معظم OS)
2. **Prevention بـ Resource Ordering** (عملي وفعال)
3. **Detection & Recovery** (قواعد بيانات، بعض الخوادم)
4. **Avoidance** (نادر جداً - افتراضات غير واقعية)

القاعدة الذهبية:

- أنظمة عامة → Ignore
- أنظمة حرجة → Prevention (Resource Ordering)
- قواعد بيانات → Detection & Recovery
- أنظمة صغيرة محددة → Avoidance

Liveness Failures

🎯 المفهوم العام: Liveness Failure

التعريف: فشل النظام في إحراز تقدم وإنجاز العمل، رغم عدم وجود أخطاء منطقية.

الأنواع الثلاثة:

```text
1. Deadlock → الخيوط محجوبة ومتوقفة تماماً
2. Livelock → الخيوط نشطة لكن بدون تقدم
3. Starvation → خيط محدد محروم من الموارد
```

📍 أولاً: Livelock (اللايف لوك)

التعريف

الخيوط **نشطة ومستمرة في التنفيذ**، لكنها **تكرر نفس المحاولات الفاشلة** دون تحقيق أي تقدم فعلي.

المقارنة مع Deadlock

الخاصية	Deadlock	Livelock
حالة الخيوط	محجوبة (Blocked)	نشطة (Active)
CPU Usage	0%	مرتفع
التقدم	لا تقدم	لا تقدم
الحركة	متوقفة تماماً	تتحرك لكن بلا فائدة

المثال التوضيحي

تشبيه الممر الضيق:

شخصان يحاولان العبور من ممر ضيق في اتجاهين متعاكسين:

- الشخص الأول يتحرك يميناً ليفسح المجال
- الشخص الثاني يتحرك يساراً في نفس اللحظة
- يتقابلان مرة أخرى
- يعكسان الحركة معاً
- يظلان "يرقصان" بدون أن يعبر أحد

هما **يتحركان** (Active) لكن **لا يتقدمان** (No Progress).

السيناريو التقني

خيطان يحاولان أخذ قفلين:

1. Thread T1 يأخذ Lock_A
2. Thread T2 يأخذ Lock_B
3. T1 يحاول Lock_B (فشل) → يحرر Lock_A ويحاول مرة أخرى
4. T2 يحاول Lock_A (فشل) → يحرر Lock_B ويحاول مرة أخرى
5. يعيدان المحاولة في نفس اللحظة → نفس النتيجة
6. التكرار إلى ما لا نهاية

الفرق عن Deadlock: هنا الخيوط **تحرر** الموارد وتحاول مرة أخرى، لكن التوقيت المتزامن يفشلها دائماً.

الحل

Random Backoff (التأخير العشوائي):

- عند الفشل، الخيط ينتظر فترة **عشوائية** قبل إعادة المحاولة
- هذا يكسر التزامن ويسمح لأحدهم بالنجاح
- نفس الآلية المستخدمة في شبكات Ethernet عند تصادم الحزم

📍 ثانياً: Starvation (المجاعة)

التعريف

خيط أو عملية **تنتظر مورداً لفترة طويلة جداً** (قد تكون لا نهائية) لأن الموارد تُمنح باستمرار لخيوط أخرى.

الفرق عن Deadlock

الخاصية	Deadlock	Starvation
النطاق	مجموعة عمليات متوقفة	عملية واحدة محرومة
النظام	متجمد جزئياً	يعمل بشكل طبيعي
العمليات الأخرى	متوقفة	تعمل وتنتهي
السبب	دورة دائرية	توزيع غير عادل

النقطة الجوهرية: في Starvation، **النظام يعمل** والعمليات **تنجز مهامها**، لكن عملية معينة هي **الضحية المنسية**.

أسباب حدوثها

1. سياسات الجدولة غير العادلة

- **Priority Scheduling**: العمليات منخفضة الأولوية قد لا تحصل على CPU أبداً
- العمليات عالية الأولوية تأتي باستمرار وتسبقها

2. في Deadlock Prevention

عند تطبيق قيود صارمة (مثل Resource Ordering)، بعض العمليات قد تُرفض طلباتها باستمرار.

3. في Deadlock Recovery

المشكلة الأساسية: عند اختيار ضحية لسحب الموارد منها:

- النظام يختار العملية الأقل تكلفة
- قد تكون نفس العملية دائماً هي الأرخص
- تُختار كضحية مراراً وتكراراً
- لن تنتهي أبداً

مثال:

- Process P1: عمل 10 دقائق حتى الآن
- Process P2: عمل دقيقة واحدة فقط
- عند الحاجة لضحية: P2 أرخص (خسارة أقل)
- في كل مرة يحدث Deadlock: يتم اختيار P2
- P2 تعود للبداية في كل مرة → Starvation

الحل

الحل العام: **Aging** (التقادم)

زيادة أولوية العملية تدريجياً كلما انتظرت أطول، حتى تحصل على فرصتها في النهاية.

الحل في **Deadlock Recovery**

تتبع عدد مرات الضحية:

- إضافة عامل "عدد مرات Rollback" في حساب التكلفة
- العملية التي تم اختيارها k مرات تصبح تكلفتها أعلى
- هذا يحميها من الاختيار المتكرر

المعادلة المحسنة:

```text
تكلفة الضحية = العمل المنجز + (عدد مرات Rollback × عامل الحماية)
```

بهذا، العملية التي عانت كثيراً لن تُختار مرة أخرى.

📊 المقارنة الشاملة

الظاهرة	حالة الخيوط	التقدم	النظام	السبب الرئيسي	الحل
Deadlock	Blocked	لا	متجمد جزئياً	دورة دائرية	كسر شرط/اكتشاف
Livelock	Active	لا	CPU عالي	تزامن فاشل	Random backoff
Starvation	Waiting	لا (للضحية)	يعمل طبيعياً	توزيع غير عادل	Aging/تتبع

🎯 الخلاصة المركزة

Livelock:

- الخيوط تعمل لكن بلا تقدم
- تكرار محاولات فاشلة بتزامن
- الحل: انتظار عشوائي

Starvation:

- عملية واحدة محرومة بينما الآخرون يعملون
- سبب شائع: اختيار متكرر كضحية في Recovery
- الحل: Aging أو تتبع عدد مرات الضحية

الارتباط بالديدلوك:

- Livelock قد ينتج من محاولات تجنب Deadlock
- Starvation قد تنتج من معالجة Deadlock (Victim selection)
- كلها Liveness Failures